

# BINV2150-A – Javascript : avancé

## Séance 5 – Sérialisation

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École
Léonard
de Vinci

# Sérialisation / Désérialisation



**Sérialisation** Transformer des données complexes en une représentation textuelle.

**Désérialisation** Reconstruire des données complexes à partir de leur représentation textuelle.

1. Communication entre applications (frontend/backend)
2. Persistance des données

# JSON (JavaScript Object Notation)



- Format léger de sérialisation de données.
- Facile à lire et écrire pour les humains.
- Facile à analyser et générer pour les machines.
- Standard permettant l'interopérabilité entre différents systèmes.

# Du client vers le serveur



- `app.use(express.json())` pour désérialiser/parser le corps des requêtes en JSON.
- `res.json(data)` pour sérialiser les données en JSON et l'envoyer dans la réponse.

# Persistance des données



- `JSON.stringify(data)` pour sérialiser les données en string JSON.
- `JSON.parse(jsonString)` pour parser un string JSON en données.
- $\rightarrow$ Stocker ces strings dans des fichiers.

# Librairie JSON

## Lire l'état de la collection



```ts
// file: utils/json.ts

function parse<T>(filePath: string, defaultArray: T[] = []): T[]
```

```ts
// usage in services :

import { parse } from "../utils/json";

const defaultItems: Item[] = [...];
const jsonDbPath = path.join(__dirname, "/../data/items.json");

const items = parse(jsonDbPath, defaultItems);
```

# Librairie JSON

## Écrire une modification de la collection



```ts
// file: utils/json.ts

function serialize(filePath: string, object: object)
```

```ts
// usage in services :

import { parse, serialize } from "../utils/json";

const defaultItems: Item[] = [...];
const jsonDbPath = path.join(__dirname, "/../data/items.json");

let items = parse(jsonDbPath, defaultItems);
// modify items ...
serialize(jsonDbPath, items);
```

# Architecture de l'application



```
project/
--- .vscode/
    --- launch.json
--- bin/
    --- www.ts
--- build/
--- data/
    --- ...
    --- items.json
--- node_modules/
--- REST Client/
    --- ...
    --- items.http
--- routes/
    --- ...
    --- items.ts
--- services/
    --- ...
    --- items.ts
--- utils/
    --- auths.ts
    --- json.ts
    --- type-guards.ts
--- .eslintrc
--- .gitignore
--- app.ts
--- package.json
--- package-lock.json
--- README.md
--- tsconfig.json
--- types.ts
```

# Résumé



- La sérialisation permet de transformer des données complexes en une représentation textuelle.
- JSON est un format populaire pour la sérialisation, facile à lire et à écrire.
- En Node.js/Express, on utilise `express.json()` pour parser les requêtes JSON et `res.json()` pour envoyer des réponses JSON.
- On peut utiliser `JSON.stringify()` et `JSON.parse()` pour sérialiser et désérialiser manuellement des données depuis et vers des strings.
- La librairie fournie `utils/json.ts` et ses fonctions `parse` et `serialize` facilitent la gestion d'un fichier de "base de données" JSON dans notre application.